

Digital Design (F215) — Master Reference Sheet

Combinational Logic (Number systems through PLDs + Verilog essentials)

1. Number Systems & Base Conversion

- A number in base r has value $\sum_i d_i r^i$ — each position is worth r times the one on its right. Binary weights (right→left) are the powers of 2 (memorise): 1, 2, 4, 8, 16, 32, 64, 128, 256, 512, 1024, ...
- n bits produce 2^n different patterns; unsigned they represent 0 to $2^n - 1$.
- To store N distinct items you need $\lceil \log_2 N \rceil$ bits (8 items → 3 bits, 10 decimal digits → 4 bits).

Binary ↔ Hex ↔ Octal (no arithmetic needed)

- $16 = 2^4 \Rightarrow 1$ hex digit = 4 bits. $8 = 2^3 \Rightarrow 1$ octal digit = 3 bits. Hex letters $A=10$ to $F=15$.
- Always group starting from the binary point**; pad the far-left group with 0s.
- Example: $1011010_2 = \underbrace{101}_5 \underbrace{1010}_A = 5A_{16}$ and $\underbrace{1}_1 \underbrace{011}_3 \underbrace{010}_2 = 132_8$.

Decimal → Binary

- Integer part** — **divide by 2 repeatedly**. The remainders are the bits, **LSB first**; read them bottom-to-top. (Dividing by 2 shifts right; the remainder is the bit that falls off.)
- Fraction part** — **multiply by 2 repeatedly**. The integer part that pops out each time is the next bit **after the point**, top-to-bottom. May never terminate (e.g. 0.1_{10}).
- General base r : divide by r for the integer part, multiply by r for the fraction. Base → decimal is always $\sum(\text{digit} \times \text{weight})$.

2. Signed Numbers & 2's Complement

- Sign-magnitude**: MSB is the sign (1 = negative), the rest is the magnitude. Has two zeros (+0, -0).
- 1's complement**: negate by inverting every bit. Still two zeros.
- 2's complement**: negate by **inverting every bit and adding 1**. One zero, and ordinary binary addition "just works" — *this is what hardware and this course use*.

Key idea: in n -bit 2's complement the MSB has weight -2^{n-1} ; all other bits are normal. Example (4-bit): $1011 = -8 + 0 + 2 + 1 = -5$. Range -2^{n-1} to $+2^{n-1} - 1$ (one *more* negative value than positive). 4-bit: $-8 \dots + 7$; 8-bit: $-128 \dots + 127$.

- Fast negate**: copy bits from the right up to and including the first 1, then invert everything to its left ($0110100 \rightarrow 1001100$).
- Sign extension** (widen a number): copy the MSB into the new high bits. $-3 = 1101$ (4-bit) = 11111101 (8-bit); padding with 0 would give +13.
- The most-negative number has no positive counterpart: negating 1000 (-8) gives 1000 again.

3. Overflow — When the Answer is Wrong

- Unsigned addition**: result is wrong $\Leftrightarrow$ carry-out $C_n = 1$ (true sum needed one more bit).
- Unsigned subtraction** (done internally as $A + B' + 1$): $C_n = 1$ means *no borrow*, $A \geq B$, result correct. $C_n = 0$ means $A < B$; the output bits are the 2's complement of $(B - A)$. The carry is an *inverted borrow*.
- Signed add or subtract**: overflow flag $V = C_{n-1} \oplus C_n$ (carry *into* the sign bit XOR carry *out* of it). Ignore C_n for the signed answer.
- Adding a positive and a negative number can **never** overflow (the result lies between them).

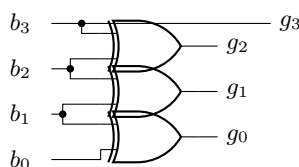
Bit-width detective (recurring exam type): some adder outputs are correct, some wrong. A wrong output = true value mod 2^n (re-read with MSB weight -2^{n-1} if signed). Use a *correct* row to lower-bound n (that value fits) and a *wrong* row to pin 2^n (how much it wrapped). E.g. $50+14$ shows $0 \Rightarrow 2^n \mid 64 \Rightarrow n \leq 6$; $50+13=63$ correct $\Rightarrow n \geq 6$; so $n = 6$.

4. Binary Codes

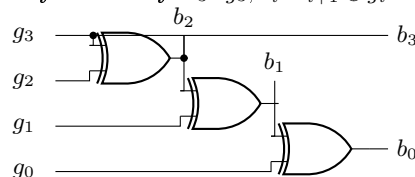
- BCD (8421)**: each decimal digit is written as its own 4-bit binary ($59 \rightarrow 0101\ 1001$). Patterns 1010–1111 (10–15) **never occur**, so in any K-map on a BCD digit they are **don't-cares** ($d(10, 11, 12, 13, 14, 15)$).
- Excess-3**: digit n is the 4-bit binary of $(n + 3)$ ($0 \rightarrow 0011$, $5 \rightarrow 1000$, $9 \rightarrow 1100$). **Self-complementing**: inverting all four bits of the code for n gives the code for $9 - n$.
- Gray code**: consecutive values differ in **exactly one bit**, including the wrap last→first. Used for rotary/position sensors (a mid-transition reading is at worst the neighbour). **Not weighted** — convert to binary to do arithmetic.
- Parity**: append a bit so the number of 1s is even (even parity) or odd. $P_{\text{even}} = d_3 \oplus d_2 \oplus d_1 \oplus d_0$. Checker XORs all received bits; 0 = ok. Detects any **odd** number of flips; two flips cancel.

Binary ↔ Gray conversion (XOR networks)

Binary → Gray: $g_3 = b_3$, $g_i = b_{i+1} \oplus b_i$



Gray → Binary: $b_3 = g_3$, $b_i = b_{i+1} \oplus g_i$



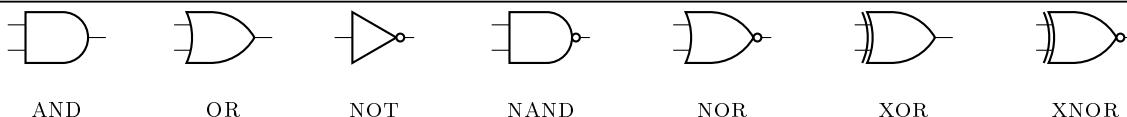
Gray-coded inputs in a K-map: write the truth-table rows in the *physical* (Gray) order first, then place each row into the map by its *binary value* (as the minterm number). Mixing the two orders is the most common mistake.

5. Boolean Algebra

Law	OR form	Law	AND form (dual)
Identity	$x + 0 = x$		$x \cdot 1 = x$
Null	$x + 1 = 1$		$x \cdot 0 = 0$
Idempotent	$x + x = x$		$x \cdot x = x$
Complement	$x + x' = 1$	Involution	$(x')' = x$
Absorption	$x + xy = x$		$x(x + y) = x$
Redundant lit.	$x + x'y = x + y$		$x(x' + y) = xy$
Distributive	$x(y + z) = xy + xz$	(2nd)	$x + yz = (x + y)(x + z)$
De Morgan	$(x + y)' = x'y'$		$(xy)' = x' + y'$
Consensus	$xy + x'z + yz = xy + x'z$		

- **Precedence:** NOT > AND > OR (so $AB' + C$ means $(A(B')) + C$). **Duality:** swap every $+$ $\leftrightarrow \cdot$ and $0 \leftrightarrow 1$; if an identity holds, its dual holds.
- **Watch:** $(AB)' = A' + B'$ *not* $A'B'$. Look for absorption before drawing gates, e.g. $pqr + q + s = q + s$ (p, r don't matter).
- **Why the tricky ones work:** $x + x'y = x + y$ because $x'y$ only matters when $x = 0$, and then $x' = 1$. $x + yz = (x + y)(x + z)$ expands to $x + xy + xz + yz = x + yz$. Consensus term yz is covered by the other two.

6. Logic Gates



A	B	AND	OR	NAND	NOR	XOR	XNOR	A'
0	0	0	0	1	1	0	1	1
0	1	0	1	1	0	1	0	1
1	0	0	1	1	0	1	0	0
1	1	1	1	0	0	0	1	0

- AND = 1 iff *all* inputs 1 (any single 0 forces 0). OR = 1 iff *any* input 1.
- XOR = 1 iff inputs *differ*; a multi-input XOR = 1 iff an **odd** number of inputs are 1 (parity). XNOR = 1 iff inputs *equal* / even number of 1s (1-bit equality).
- **Controlling-value tricks** (used constantly): $A \cdot 1 = A$, $A \cdot 0 = 0$ (AND = enable/block); $A + 0 = A$, $A + 1 = 1$; $A \oplus 0 = A$, $A \oplus 1 = A'$ (XOR = **programmable inverter** — basis of the adder/subtractor).

7. Minterms, Maxterms & Canonical Forms

- Row i of the truth table is the input combination whose binary value is i , with the **first-named variable as the MSB**. For $F(A, B, C)$, row 5 = 101 is $A=1, B=0, C=1$.
- **Minterm** m_i : AND of all variables; a variable that is **0** in that row is **primed**. Equals 1 at row i only. $m_5 = AB'C$.
- **Maxterm** M_i : OR of all variables; a variable that is **1** in that row is **primed**. Equals 0 at row i only. $M_5 = A' + B + C'$. Note $M_i = (m_i)'$.
- **Canonical SOP** = $\Sigma m(\text{rows where } F=1)$. **Canonical POS** = $\Pi M(\text{rows where } F=0)$. The two lists are complementary. Also $F' = \Sigma m(0\text{-rows})$.
- A product term missing a variable covers several rows: $AB' = AB'C + AB'C'$ ($m_4 + m_5$). Each missing variable doubles the minterms covered.
- **Standard (2-level) forms:** SOP = a layer of ANDs feeding one OR; POS = a layer of ORs feeding one AND. At most 2 gate delays — why SOP/POS are preferred.

8. Karnaugh Maps

Fold the truth table so neighbours differ in one bit; then simplification is just drawing rectangles. Headers go in **Gray order** 00, 01, 11, 10.

Example groups (wrap & corners)

4-variable cell numbering				
$AB \backslash CD$	00	01	11	10
00	0	1	3	2
01	4	5	7	6
11	12	13	15	14
10	8	9	11	10

$AB \backslash CD$	00	01	11	10
00	1	0	0	1
01	0	1	1	0
11	0	1	1	0
10	1	0	0	1

$B'D'$

BD

- **Why it works:** two minterms differing in one variable merge and that variable drops ($AB'C + ABC = AC$). A group of 2^k cells eliminates k variables.
- **Legal group:** a rectangle of 1, 2, 4, 8, 16 cells, may **wrap across edges and the four corners**, containing only 1s (and chosen Xs). Make groups as **large** and as **few** as possible.
- **Reading a group:** variables that stay constant survive ($0 \rightarrow$ primed, $1 \rightarrow$ plain); variables that change vanish. (Four corners = $B'D'$.)
- **PI** = a group that cannot be enlarged. **EPI** = a PI covering a 1 that *no other PI* covers — must be in every answer.
- **Procedure:** (1) find all PIs; (2) take the EPIs; (3) cover the remaining 1s with the fewest, largest extra PIs.
- **Don't-cares (X):** use an X as 1 *only if it enlarges a group*; decide each X independently; never form a group of only Xs; an X never makes a PI essential.
- **POS from a K-map:** group the **0s** to get a minimal SOP of F' , then apply De Morgan to each term. With don't-cares, the minimal SOP and minimal POS may differ as functions.
- **Cyclic map:** no EPIs exist $\Rightarrow$ several equally-minimal answers (list all if asked).

- **5 variables:** draw two 4-variable maps ($A=0$ and $A=1$); a cell is also adjacent to the same cell in the other map. A group appearing in the same place in both maps loses A .

9. Quine–McCluskey (tabular method)

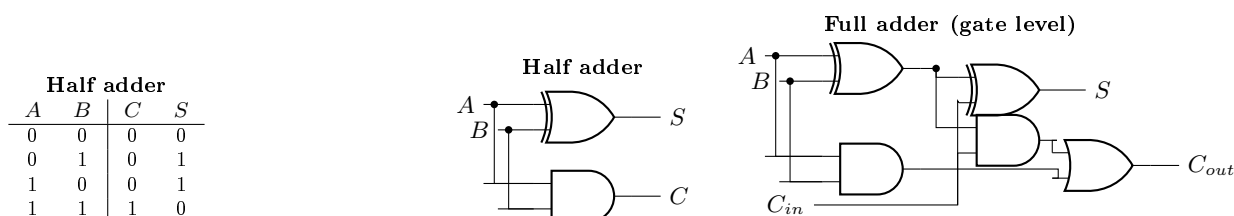
Same merges as a K-map but done on bit strings, so it scales to any number of variables.

- List all minterms **and don't-cares** in binary, grouped by their number of 1s.
- Compare adjacent groups: any pair differing in exactly one bit merges to a term with a dash '–' at that position; tick both parents. Only merge terms whose dashes are already in the same places.
- Repeat on each new column until nothing merges. **Unticked terms are the prime implicants.**
- **PI chart:** rows = PIs, columns = the required minterms **only** (exclude don't-cares). A column with a single $\times$ makes that PI **essential**. Cover the rest with the fewest remaining PIs.
- Dash string $\rightarrow$ literals: a dash = variable gone, 1 = plain, 0 = primed ($0-01 = A'C'D$).

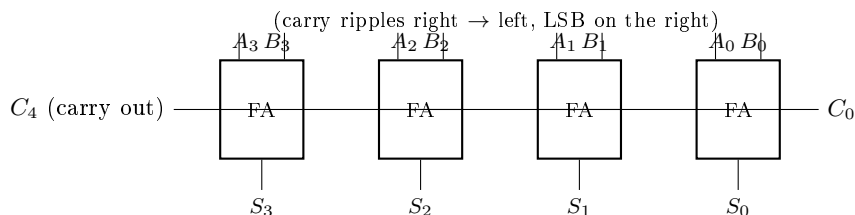
10. NAND-only / NOR-only & Bubble Pushing

- Both are **universal**. From NAND: NOT = 1, AND = 2, OR = 3 gates (OR = 1 if complemented inputs are free), XOR = 4. From NOR: NOT = 1, OR = 2, AND = 3, XNOR = 4.
- NOT: tie the two inputs together. AND = $((AB)')'$. OR = $(A'B')'$ (De Morgan).
- **SOP $\rightarrow$ NAND-NAND:** put two bubbles on each middle wire (two inversions cancel). Each AND becomes a NAND; the OR with bubbled inputs *is* a NAND. Same gate count as AND-OR.
- **POS $\rightarrow$ NOR-NOR:** the exact dual (ORs $\rightarrow$ NORs, final AND with bubbled inputs $\rightarrow$ NOR).
- **Single-literal term trap:** e.g. $F = AB + C$. The C term has no first-level gate, so the second-level NAND must receive C' (use the complemented input, or add a NAND-inverter).
- **Asked for NAND?** minimise the **SOP** (group 1s). **Asked for NOR?** minimise the **POS** (group 0s). For "2-input gates only": factor first to cut fan-in and share sub-terms, then push bubbles, then count.

11. Adders



- **Half adder:** $S = A \oplus B$, $C = AB$. (Adds two bits; cannot take a carry-in.)
- **Full adder:** $S = A \oplus B \oplus C_{in}$, $C_{out} = AB + AC_{in} + BC_{in} = AB + C_{in}(A \oplus B)$ ($C_{out} = \text{majority}$). The S K-map is a checkerboard, so no AND/OR simplification — use XORs.
- **FA from two half adders + OR:** $HA1(A, B) \rightarrow (s_1, c_1)$; $HA2(s_1, C_{in}) \rightarrow (S, c_2)$; $C_{out} = c_1 + c_2$. The two carries are never 1 together, so OR (or XOR) works.
- **Ripple-carry adder:** chain n full adders, each $C_{out} \rightarrow$ next C_{in} . Worst-case delay = $2n + 1$ gate levels ($O(n)$); a 64-bit adder = 129 delays. Carry must settle before the next edge.

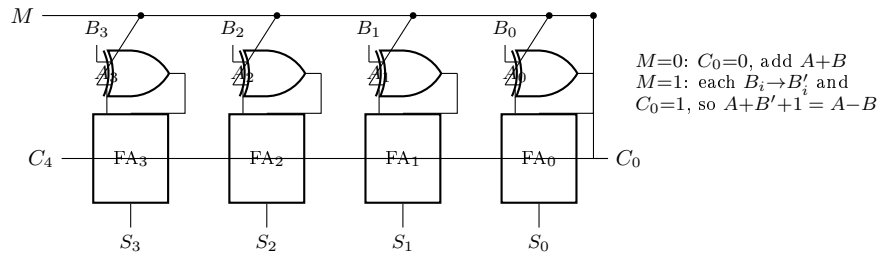


12. Subtractors

Half subtractor ($X - Y$)			
X	Y	B (borrow)	D (diff)
0	0	0	0
0	1	1	1
1	0	0	1
1	1	0	0

- **Half subtractor:** $D = X \oplus Y$, $B = X'Y$. Note B is **not symmetric** ($X'Y \neq Y'X$): swapping X, Y changes the borrow.
- **Full subtractor:** $D = X \oplus Y \oplus B_{in}$, $B_{out} = X'Y + X'B_{in} + YB_{in} = X'Y + (X' + Y)B_{in}$.
- **FS from two half subtractors + OR:** the second HS must compute $(d_1 - B_{in})$, i.e. ports $.x(d_1), .y(B_{in})$. Wiring it $.x(B_{in}), .y(d_1)$ is the classic bug (fails at $X=Y=0, B_{in}=1$: D still right since XOR is symmetric, B_{out} wrong).
- **Blocks used as gates:** HS with $X=1 \Rightarrow D = Y'$ (free inverter), $B=0$. FS with $X=0 \Rightarrow B_{out} = Y + B_{in}$; with $X=1 \Rightarrow B_{out} = Y \cdot B_{in}$.

13. Adder/Subtractor & Carry-Lookahead



- **Adder/subtractor:** XOR every B_i with the mode line M (XOR = controlled inverter), and feed M into C_0 . $M=0$ computes $A+B$; $M=1$ computes $A+B'+1 = A-B$. Overflow $V = C_{n-1} \oplus C_n$ (works unchanged for subtraction).
- **Carry-lookahead** removes the ripple. Define **generate** $G_i = A_i B_i$ and **propagate** $P_i = A_i \oplus B_i$; then $C_{i+1} = G_i + P_i C_i$ and $S_i = P_i \oplus C_i$. Substitute to make every carry a flat 2-level function:

$$C_1 = G_0 + P_0 C_0, \quad C_2 = G_1 + P_1 G_0 + P_1 P_0 C_0, \quad C_4 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_0.$$

- Delay becomes a constant **4 gate levels** for any n (P/G, then 2-level carry, then sum XOR); gate count grows $O(n^2)$ and fan-in becomes impractical, so real adders use 4-bit CLA blocks. Hierarchical (block-generate/propagate) lookahead gives $O(\log n)$ delay with $O(n)$ gates.
- For the *carry*, P_i may be $A_i + B_i$; for the *sum* it must be $A_i \oplus B_i$. **Borrow-lookahead:** $G_i = X'_i Y_i$, $P_i = X'_i + Y_i$ (OR-type).

14. BCD Adder & Code Converters

- **BCD adder:** add two BCD digits + C_{in} with an ordinary 4-bit binary adder giving $K Z_8 Z_4 Z_2 Z_1$. If the sum exceeds 9, the result must skip the six unused codes, so add 0110 (**6**) and raise the decimal carry.
- **Correction needed** $C = K + Z_8 Z_4 + Z_8 Z_2$. Feed $0 C C 0$ ($= 6$ when $C=1$) into a second 4-bit adder; C is the decimal carry-out. An n -digit adder is n such stages, carry to carry.
- **Code converter (general procedure for any multi-output circuit):** count inputs/outputs; build one truth table (mark impossible inputs as X); draw **one K-map per output**; share gates between outputs. BCD $\rightarrow$ Excess-3: $z = D'$, $y = (C \oplus D)'$, etc., using the 10–15 don't-cares.

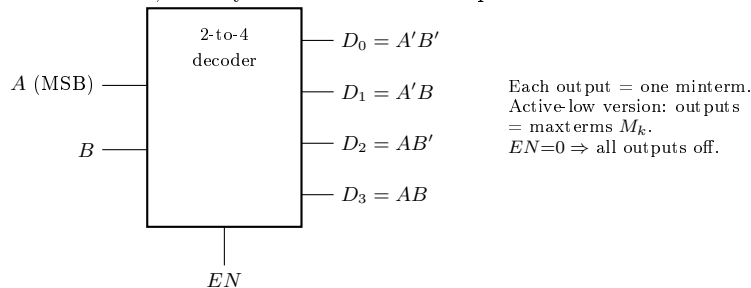
15. Multipliers & Magnitude Comparator

- A **partial product** is the multiplicand ANDed with one multiplier bit, shifted left by that bit's position; sum the partial products.
- 2×2 : $C_0 = A_0 B_0$; C_1 = half-add of $A_1 B_0$ and $A_0 B_1$; C_2 = half-add of $A_1 B_1$ and the carry from C_1 ; C_3 = its carry. $J \times K$ **multiplier:** $J \cdot K$ AND gates, $(J-1)$ K -bit adders, product is $J+K$ bits.
- **Multiply by a constant:** $\times 6 = \times 4 + \times 2$ = (shift left 2) + (shift left 1); one adder per extra 1 in the constant.
- **Comparator:** equal-bit $x_i = (A_i \oplus B_i)'$ (XNOR). Then $A=B$ is $x_3 x_2 x_1 x_0$; and (dictionary order from the MSB)

$$A > B = A_3 B'_3 + x_3 A_2 B'_2 + x_3 x_2 A_1 B'_1 + x_3 x_2 x_1 A_0 B'_0, \quad A < B \text{ symmetric (swap } A, B).$$

16. Decoders

A decoder expands an n -bit code into 2^n lines, exactly one active. Each output line is a minterm.



- n inputs $\rightarrow 2^n$ outputs; built from 2^n AND gates (each a minterm) plus inverters, plus an optional **enable** into every gate.
- **Active-high** outputs are minterms; **active-low** (bubbled) outputs are maxterms M_k . A decoder with enable is a **demultiplexer** (put data on EN).
- **Build a bigger decoder** from smaller ones: the MSB(s) drive the enables (3-to-8 = two 2-to-4, enabled by A' and A).
- **Implement any F with one decoder:** choose the cheapest of — active-high + **OR** of the 1-rows; active-high + **NOR** of the 0-rows; active-low + **NAND** of the 1-rows; active-low + **AND** of the 0-rows. Pick the gate with min($\#1$ s, $\#0$ s) inputs. One decoder can drive several functions (an OR per function); a single needed minterm needs no gate at all.

17. Encoders & Priority Encoders

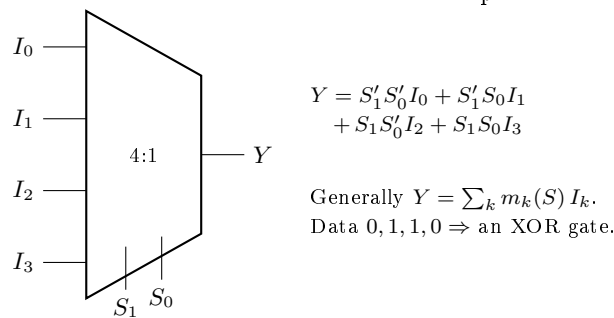
- The reverse of a decoder: 2^n inputs $\rightarrow$ the n -bit index of the active line. A **plain encoder** is just OR gates:

$$Y_2 = D_4 + D_5 + D_6 + D_7, \quad Y_1 = D_2 + D_3 + D_6 + D_7, \quad Y_0 = D_1 + D_3 + D_5 + D_7.$$

- Two problems: if *two* inputs are active the output is the OR of their codes (garbage); "no input active" is indistinguishable from " D_0 active" (both give 000).
- **Priority encoder** fixes both: the highest-index active input wins, plus a **Valid** output. 4-to-2: $x = D_2 + D_3$, $y = D_3 + D_1 D'_2$, $V = D_0 + D_1 + D_2 + D_3$. ($y=1$ for index 3, or index 1 only if the higher-priority D_2 is off.)
- **Decoder $\rightarrow$ wires $\rightarrow$ encoder** = any one-to-one code converter: wire decoder line k to encoder input $f(k)$ (e.g. bit-reversal, binary $\rightarrow$ Gray). The mapping must be one-to-one.

18. Multiplexers & Demultiplexers

A multiplexer is a digitally-controlled switch: the select lines choose which data input reaches the output.



- 2^n data inputs, n select lines. 2:1: $Y = S'I_0 + SI_1$.
- **Implement F with a MUX — three methods:**
 - All n vars on the selects: tie each data input I_k to that truth-table row's output (0 or 1). The mux is a truth-table lookup.
 - $n-1$ vars on the selects (one var, say D , left over): each select combination covers two rows; the data input is one of $\{0, 1, D, D'\}$. Build the **implementation table**: write minterms in a D' row and a D row, circle the 1s, read each column.
 - $n-2$ vars on the selects: each data input is a function of the two leftover variables — K-map that column (use don't-cares).
- **Order matters:** " AB as s_1s_0 " means A is the MSB select. Only-true-inputs trick: feed the leftover variables to a decoder to get complemented product terms for free.
- **Demultiplexer:** one data line routed to one of 2^n outputs = a decoder with the data on its enable.

19. Programmable Logic (ROM / PLA / PAL)

Every SOP circuit is an AND array (products) feeding an OR array (sums); PLDs pre-build that with programmable crossings (an $\times$ = a connection).

Device	AND array	OR array	Think of it as
ROM	fixed (full decoder, all 2^n minterms)	programmable	a stored truth table ($2^n \times m$)
PLA	programmable	programmable	shared minimal SOP
PAL	programmable	fixed (own ANDs per output)	separate minimal SOPs

- **PLA size** = inputs $\times$ #product terms $\times$ outputs. "Minimum-size PLA" means the **fewest distinct product terms** across all outputs — a term needed by two outputs is built once and shared (sometimes it is worth using a *non-prime* term that both outputs can reuse).
- **PAL** has no sharing (each OR has its own ANDs): minimise each output separately and check it fits the product-terms-per-output limit.
- **Programming table** (one row per product term): input column 1 = true literal used, 0 = complemented literal used, – = variable unused; output column 1 = this term feeds that output's OR gate.
- **ROM:** n address lines $\rightarrow$ decoder makes all 2^n minterms (word lines); programming = choosing which outputs get a 1 for each word (= writing the truth-table output columns).

20. Verilog Essentials

- Verilog *describes* hardware; statements are **concurrent**, line order does not matter. Gate primitive lists the **output first**: `and(y,a,b)`, `xor(s,a,b)`.
- **wire** = a physical connection, driven continuously (a gate output, an **assign**, or a module output); it *cannot* be assigned inside **initial/always**. **reg** = holds its last value; used for anything assigned in **initial/always** (not necessarily a hardware register).
- **Testbench rule:** signals you drive from **initial** are **reg**; the DUT's outputs you observe are **wire**.
- Dataflow: `assign y = s ? i1 : i0;` (a 2:1 mux); `assign {cout,sum} = a + b + cin;` (concatenation catches the carry).
- Behavioural: `always @(*)` = combinational (use blocking `=`; assign every output on every path or a **latch** is inferred). `always @(posedge clk)` = flip-flops (use non-blocking `<=`).
- **Blocking** = updates immediately (like software); **non-blocking** `<=` evaluates all right-hand sides first, then updates together (models all flip-flops sampling at one edge). Arithmetic wraps to the destination width.
- Operators: `~a` bitwise vs `!a` logical; `&a` / `^a` are reductions (`^a` = parity). `$display` prints once; `$monitor` prints whenever an argument changes; `#n` delays; `$finish` ends, `$stop` pauses.

Open-book exam checklist: (1) write the truth table *before* the K-map and mark the don't-cares; (2) label K-map axes with variable names in Gray order and re-check which letter is the MSB; (3) for "use only these components", first write what each allowed block computes, then map the pieces of F onto them; (4) in state/answer tables use the *exact* column order and notation the question gives — marking is often all-or-none, so one wrong cell or a swapped select line costs the whole part.